

Discrete Semantic State Representations for Efficient Policy Learning

Rohan Menon

Vrije Universiteit Amsterdam, De Boelelaan 1105, 1081 HV Amsterdam

Abstract. This thesis investigates the extent to which discrete, vector-quantised (VQ) representations learned by a Transformer encoder align with semantically meaningful environment states in a simple reinforcement learning environment. Currently existing discrete world models are evaluated primarily on downstream tasks such as reward and sample efficiency. This creates the open question of whether **individual codebook tokens** correspond to interpretable concepts such as “the door is open” or “the agent is carrying the key”. In order to answer this question, an object-centric VQ encoder is trained on 60k observations from **MiniGrid-DoorKey-5x5** using reconstruction loss only, no reward signal or state labels are provided. **Which helps to examine if the encoder can identify specific semantic concepts without supervision.** The encoder uses a base Transformer model with three separate VQ heads which target the agent, key and door object specifically. In order to evaluate semantic alignment between **these** codes and distinct semantic states logistic regression probes are run on ground-truth environment labels. The key head achieves 0.997 probe accuracy for the **carrying_key label**, with four specific codebook tokens exhibiting perfect accuracy in identifying the token ($P(\text{carrying_key} = \text{True}) = 1.000$), demonstrating clear semantic delineation. The door head similarly is also able to semantically **seperate** the open/locked state with a single dedicated token. In order to use these tokens and evaluate performance/sample efficiency a downstream **PPO** policy was trained on the fully discrete representation **(OC-All-Disc)** and achieved a mean return of **0.768 ± 0.158 across five seeds training over 200k environment steps**, with all seeds converging, compared to negligible result when spatial information is missing. The results of the PPO policy trained on these discrete codes suggest that object-centric codebook discretisation promotes semantically grounded discrete representations, though the approach does not improve sample efficiency or create increased training stability over raw pixel baselines in this constrained setting.

Keywords: Reinforcement Learning · Discrete Representation Learning · Vector Quantization · Object-Centric Representations · Semantic Interpretability

1 Introduction

1.1 Context and Motivation

Deep reinforcement learning (RL) has proven to be able to produce agents that are capable of strong performance across a variety of challenging domains, from Atari games [15] to complex robotic manipulation [11]. However, this performance comes with the consequence that agents trained directly from high-dimensional observations require millions of interactions with the environment to learn reliable structure and encode knowledge of their environment. This **significant** sample inefficiency represents a **key** challenge for RL algorithms especially in environments where data collection is expensive or unsafe.

Therefore, a parallel research line proposes to address this sample inefficiency in RL environments by first learning a compact latent representation of the environment, over which a policy can be trained more efficiently. Hafner et al. [3, 4] introduced Dreamer, which learns a world model in a continuous latent space and uses it to simulate future experience, **significantly improving** sample efficiency over model-free methods and establishing latent-space planning as a compelling alternative to traditional RL.

Additionally, research in cognitive science has proposed another reason for structured representations. According to research by Fang and Sims [2], humans use their own effective coding methods to condense continuous sensory data into small, reusable structures in order to create generalizable internal discrete representations in the brain. The idea that discrete representations could provide the basis for generalization in biological systems is supported by this research. These two concepts create a clear research gap for the improvement of robust RL systems. While cognitive research suggests that discrete, internal semantic structures may be a characteristic of generalizable learning, world models show that learning over a latent space may increase the sample efficiency of agents. Therefore, combining these two concepts to study agents that learn on discrete encoded relationships whose latent representations carry semantic structure is a natural progression motivated by these areas of research.

The extent to which learned tokens encode semantically meaningful states is **an important** aspect of discrete representations that is often overlooked despite **advancements in the field**. The **substance** of learnt codebooks is rarely examined, and existing discrete world models are typically evaluated on **performance on downstream performance factors**, such as reward and sample efficiency. This leaves the question unanswered as to whether tokens **can** consistently correspond to interpretable state ideas such as “the door is open” or “the agent is carrying the key.” **There is a variety of reasons as to why this is an important idea.** Firstly, one of the primary advantages to using discrete representations is their interpretability, tokens that do not align with **existing** semantic concepts offer limited advantage over more opaque continuous representations. **The** growing requirement for trustworthy and explainable AI [19] makes representation interpretability an important **criteria** when considering system design. Additionally, semantically grounded tokens may also be **useful when** combined with more logic

Here you refer to related work (e.g. something is overlooked). First mention the importance of the thing being overlooked, then mention it is overlooked in the next paragraph about Van den Oord et al. etc. Specifically mention there which key works left the gap open

based algorithms or neurosymbolic approaches. Where the discrete structure can be directly exploited, or used as primitives for downstream logical reasoning or planning. The availability of existing discretization techniques, powerful Transformer architectures and increased research into latent modelling creates an opportunity to investigate whether semantically aligned discrete representations can be learned from RL observations without explicit supervision.

Van den Oord et al. [20] introduced the Vector-Quantised Variational Autoencoder (VQ-VAE), demonstrating that neural networks can learn finite, discrete codebooks that reliably compress complex visual inputs, with subsequent work extending the approach to hierarchical generation [16] and simplified quantization schemes [12]. The applicability of VQ-style discretization to RL environments has been demonstrated by Micheli et al. [13], whose IRIS model, a Transformer world model operating over discrete observation tokens, achieves state-of-the-art sample efficiency on the Atari 100k benchmark. What these methods have in common is evaluation by task reward alone; the semantic content of the learned vocabulary is not examined. This thesis addresses that gap directly.

1.2 This Thesis

This thesis explores how a transformer-based encoder can be trained to create a codebook of discrete, semantically inspired tokens derived from interactions with a continuous RL environment. The goal is to analyze how well these learned representations correspond with semantically relevant states of the environment entirely without supervision. The encoder is solely trained on reconstruction and therefore, does not obtain any reward signal or particular semantic labels. This means that any learned semantic alignment must arise solely from the inherent structure of the task environment.

Main research question:

To what degree can discrete, transformer-based state encoders learn semantically meaningful abstractions that improve sample efficiency and training stability in reinforcement learning environments?

Sub-questions:

- To what degree does a VQ-style Transformer encoder learn stable, reusable discrete tokens from continuous visual input?
- To what extent do these tokens correspond to semantically meaningful environment states?
- To what extent does planning over discrete tokens reduce sample complexity compared to planning in continuous latent space?
- To what extent does tokenization reduce planning horizon complexity or support generalization?
- How interpretable are the learned tokens?

2 Related Work

This chapter examines three relevant research bodies that this thesis is based upon and frames this current contribution in relation to prior work. The aim

is to outline where existing methods succeed and where the specific question of semantic alignment within discrete RL representations has been left open.

2.1 Discrete Representation Learning

The foundational addition to the field of discrete learning, and therefore discrete representation learning is the Vector-Quantised Variational Autoencoder (VQ-VAE) of van den Oord et al. [20]. VQ-VAE proposed replacing the continuous latent space of a standard VAE with a learned finite codebook of embedding vectors. During the forward pass, each encoder output is mapped to the nearest codebook entry. This is then used as input to the subsequent decoder. Because this nearest neighbour lookup operation is non-differentiable, gradients cannot flow through this step. In order to overcome this a straight-through estimator is used, where gradients that arrive at the quantised representation are sent directly back to the encoder output, skipping the discontinuous lookup process.

However, a persistent challenge in VQ-VAE training is codebook collapse. This is when only a small subset of codes receive gradient updates, leaving the majority of codes in the codebook unused and the overall representational capacity underutilised. The original paper’s Appendix A proposed an Exponential Moving Average (EMA) update rule that separates the codebook updates from the initial optimisation step. Each codebook vector is updated as a running average of encoder outputs assigned to it rather than purely through gradient descent. Additionally, a complementary dead-code restart mechanism reinitialises underused entries by copying the embedding of a randomly selected active code, amending the codebook to encourage coverage. Together these two methods have been proven to reduce collapse, therefore resulting in increased utilisation of the entire codebook. VQ-VAE-2 [16] extent this approach to more hierarchical multi-scale representations, which enabled the creation of high-quality image generation. More recently, Finite Scale Quantization (FSQ, [12]) propose a new method that avoids explicitly maintaining the codebook entirely and bounds each dimension of the latent to a small integer grid, and relying on this implicit discretization. FSQ proposes trading codebook flexibility for increased simplicity during training and demonstrates comparable reconstruction results. Earlier work on disentangled continuous representations, particularly β -VAE [7], also act as a basis for this thesis by outlining that architectural biases can encourage structured, interpretable latent spaces unsupervised, an insight that motivates the object-centric factorisation proposed here. However the key distinction is that in all of this work, evaluation is measured based on reconstruction quality and generative fidelity. Whether or not the learned codes align with semantically meaningful concepts is yet to be examined.

great!

2.2 World Models and Latent-Space Policy Learning

The Dreamer family of world models [3, 4] showed that model-based RL functioning in a learned latent space can significantly improve sample efficiency compared to model-free approaches. DreamerV1 [3] develops a Recurrent State Space

Model (RSSM) featuring continuous stochastic latents and fully trains a policy through imagined rollouts, reaching competitive results on DMControl with significantly fewer interactions with the environment. DreamerV2 [4] improves on this finding by integrating discrete categorical latents with continuous recurrent states, showcasing competitive performance in Atari at 200k steps. DreamerV3 [5] extends its application to various domains by ensuring stable training across different scales; although it includes discrete elements, it is not solely a discrete model and is thus referenced here for thoroughness rather than for direct comparison.

The most relevant model-based research is IRIS [13], which integrates a VQ tokeniser with a Transformer world model, attaining state-of-the-art results on the Atari 100k benchmark. IRIS shows that discretization enhances RL beyond just compression; the organized vocabulary allows for efficient Transformer modeling of transition dynamics. TD-MPC2 [6] adopts a complementary strategy, acquiring a latent world model for continuous control through temporal difference learning without using explicit quantization. The previous Model-Based RL for Atari [9] demonstrated that acquired environment models can enhance sample efficiency in settings with discrete actions. Sequence modeling methods like Decision Transformer [8] utilize Transformer architectures in offline RL, whereas generalist agents [17] showcase cross-task generalization through token-based representations. In all of these approaches, assessment is carried out solely on task reward and sample efficiency indicators. The internal organization of the acquired representations, particularly, if separate tokens relate to significant environmental states, is never assessed. This thesis fills that gap by applying a linear probe methodology directly to the VQ codebooks within the controlled RL environment.

2.3 Interpretability of Learned Representations

Alain and Bengio [1] introduced the application of linear classifiers for evaluating the semantic content of intermediate representations in neural networks, demonstrating that the performance of a logistic regression probe, trained on the activations of a layer, offers a reliable estimate of the information encoded by that layer. This "probing" approach has since turned into a standard technique in representation evaluation. In this context, logistic regression examines one-hot token indices to determine if the discrete codebook accurately reflects the true environment labels without any supervised labeling during training.

A related area of research aims to create interpretable representations by design. Concept Bottleneck Models [10] require that intermediate representations align with predefined human-understandable concepts, allowing for intervention during testing. Although this method requires labeled concepts during training, it demonstrates that aligning representations with semantic ideas is feasible and beneficial in practice. Doshi-Velez and Kim [19] present a comprehensive framework for assessing interpretability in machine learning, differentiating between application-grounded, human-grounded, and functionally-grounded evaluation

standards; the linear probe approach utilized in this thesis is classified under the functionally-grounded criteria.

This thesis is unique from the aforementioned works by utilizing probe-based analysis specifically for VQ codebooks within a reinforcement learning framework, focusing not on whether a continuous representation contains semantic information but rather on whether the discrete codebook indices retain such information post-quantization. Additionally, the object-centric factorization; distinct VQ heads for every tracked object, introduces an architectural inductive bias aimed at semantic alignment that, to the authors’ knowledge, has not been previously assessed with linear probes in a reinforcement learning context. No previous studies have methodically investigated if specific VQ codes in a reinforcement learning encoder relate to semantic states of the environment, or if object-centric codebook factorization enhances this correlation. This thesis tackles both questions.

clear gap!

3 Methodology

3.1 Environment and Data

This proposed architecture was tested in MiniGrid-DoorKey-5x5 [14], a simple grid-world environment in which an agent must complete a sequential set of tasks: locate a key, pick it up, use it to unlock a locked door, and navigate to the goal cell. This specific environment was chosen for two primary reasons. Firstly, it provides access to ground-truth boolean state labels (`door_open`, `door_locked`, and `carrying_key`), that are not derivable from raw pixel values themselves. These are not shown to the agent during training to enforce the unsupervised nature of the encoding, however, they are used to directly assess semantic alignment. Secondly, the environment is small enough to run a comprehensive ablation study under a fixed computational budget while still containing enough of a non-trivial structure that requires multiple sub-goals.

The observation space is a $7 \times 7 \times 3$ integer-valued grid, representing a partial view centred on the agent. Each of the 49 cells encodes an object type, colour and state in three integer channels. The agent is not able to observe the entire 5×5 environment directly, as cells outside it’s view are masked. The action space consists of seven discrete actions: turn left, turn right, move forward, pick up, drop, toggle, and done. The reward function is very sparse, the agent receives a reward only when reaching the goal cell, this is then discounted by the number of steps taken. All the other transitions yield no reward.

a visualisation would be helpful here

To gather training data for the encoder a uniformly random policy was run for 300 episodes of up to 100 steps each, yielding 59,697 observations. This random policy data is specifically chosen to provide a variety of environment states without directly exploiting the task structure or providing any implicit supervision. Ground-truth labels (`door_open`, `door_locked`, `carrying_key`) are recorded at each step from the environment state and are used exclusively for semantic alignment evaluation; they are never provided to the encoder during training.

3.2 Encoder Architecture

The encoder follows an object-centric design. There is a shared transformer backbone which process the full observation, three independent VQ readout heads each quantise a different aspect of the scene. This factorisation is the central architectural addition and is motivated by the hypothesis that linking dedicated codebook capacity to individual objects can help promote semantically grounded tokens.

The encoder receives the $7 \times 7 \times 3$ observation as input which is flattened to 49 cells. Each of the cells three integer channels (object type, colour, state) are then mapped through separate learned embedding tables to produce a per-cell feature vector. This is then projected into a $D_{\text{model}} = 64$ dimensional space. There is a flat cell-index positional embedding which maps position indices 0-24 to a series of 32 dimensional trainable vectors, this is added to each cell representation within the 5×5 grid view. A learnable CLS token is then added to the beginning of the sequence. This results in a sequence of 50 vectors (CLS + 49 cells), which is processed by a standard transformer encoder with 2 layers, 4 attention heads and $D_{\text{model}} = 64$ throughout.

The three VQ readout heads operate as follows. The agent head takes the CLS token output and passes it through a VQ layer with $K = 64$ codebook entries, each of dimension 64, producing an embedding e_{agent} . The key head takes the Transformer output at the spatial cell corresponding to the key's current grid position and applies a VQ layer with $K = 16$, yielding e_{key} . The door head similarly reads the output at the door's cell with $K = 8$, yielding e_{door} . When an object is absent from the partial observation, for instance, when the key has been picked up and is no longer visible on the floor, the corresponding head falls back to a learned absent embedding rather than an arbitrary cell output. This fallback is itself a reliable binary signal: the key head's absent-embedding activation is tightly correlated with the `carrying_key` state.

The three VQ heads work by the following process. The agent head takes the CLS token and feeds it into a VQ layer with $K = 64$ codebook entries, each of dimension 64, producing an embedding e_{agent} . Additionally, the key head takes the transformer output at the spatial cell corresponding to the key's current position in the grid and applies a VQ layer where $K = 16$, resulting in e_{key} . Similarly, the door head applies the same process at the door's cell with $K = 8$, yielding e_{door} . In the case where an object is missing from the partial observation, such as when the key has already been picked up and is no longer visible on the floor, the corresponding head falls back to a learned absent embedding.

Each VQ layer implements the following operations. Given an encoder output z , the layer computes the L_2 distance to all K codebook entries and selects the nearest:

$$k^* = \arg \min_k \|z - e_k\|^2. \quad (1)$$

The vector quantised output is e_{k^*} , and a straight through estimator is used to propagate gradients through the discrete non-differentiable space. The backward pass treats the lookup process as an identity function. Codebook entries

a diagram of the architecture would be most helpful here. You can indicate the components with numbers and refer to these in the text

there are too many details here. Provide the method in which these were established and summarize parameters in a table. Put parameters that are not crucial to answering the RQ but that are important for reproducibility in the appendix if you run out of space

are updated using EMA, where the numerator keeps track of a running sum of encoder outputs assigned to each entry. The denominator simply tracks the number of assignment counts, the entry is then replaced by this normalised ratio. The entries where the assignment count falls below a certain threshold (`dead_code_threshold` = 1.0) are restarted by copying the embedding of a randomly chosen embedding that is active, preventing codebook collapse. The commitment loss $\beta \|z - \text{sg}(e_{k^*})\|^2$ (with $\beta = 0.10$) encourages encoder outputs to remain close to their assigned codebook entry.

To apply this to effective policy learning a discrete `PositionEmbeddingEncoder`, maps the agent’s current cell index (0 - 24, for the 5×5 grid) to a 32 dimensional trainable vector. This component does not involve a VQ layer as the position space is already discrete and therefore does not involve quantisation. This creates the full policy input for the OC-All-Disc condition where the four components: e_{agent} (64-dim), e_{key} (64-dim), e_{door} (64-dim), and e_{pos} (32-dim) are concatenated. Resulting in a 224-dimensional vector that is fed to the policy network to act over.

The encoder also has a reconstruction decoder that reconstructs all the 49 cells from the concatenated VQ embeddings, it also has per-head decoders for the key and door cells. These decoders are used strictly during pre-training and removed at policy learning. The reconstructive objective acts as the sole training information for the encoder.

3.3 Encoder Pre-Training

The encoder is trained with the random-policy dataset of 59,697 observations mentioned in Section 3.1. The goal during training is to minimize cross-entropy reconstruction loss across all 49 cells and three channels for each cell. No reward signals, environmental labels, or task-specific information are utilized. Therefore, any semantic separation that arises is derived from the structure of the observations themselves. The encoder employs Adam as the optimizer ($\text{lr} = 3 \times 10^{-3}$) for 80 epochs, utilizing a batch size of 128. EMA codebook updates and dead code restart techniques are utilized during every forward pass as mentioned in Section 3.2

An initial approach used gradient-based codebook updates (this contained a learnable embedding layer updated by backpropagation) and a single CLS-token head with $K = 32$. This approach suffered significant codebook collapse, after training there were just 11 of 32 codes active. This mechanism is a common feedback loop characteristic generated by VQ training as popular codes attract from assignments, receive more gradient updates and therefore become more central, while unused codes drift further from the distribution. The skewed distribution of random-policy observations exaggerates this effect. Switching to EMA updates with an increased codebook size increased the utilisation of the codebook codes to 57/64, and using the object-centric three-head design with codebooks sizes of $K = 64, 16$, and 8 achieved full codebook utilisation on the key and door heads (100% active) and 62% utilisation on the agent head. The full training

history of this progression is detailed in Section 4.1; hyperparameters are listed in Appendix A.

3.4 Policy Conditions

This section contains the policy conditions that were investigated in this study. The specific hyperparameters used for reproducibility are included in Appendix A. Five conditions are considered, ranging from the raw-observation baseline where the flattened observations are fed into a flat MLP to the proposed fully discrete representation. Each condition uses the same PPO training configuration [18] with a frozen encoder, the encoder weights are frozen after pre-training and not updated during policy learning. The network used by the PPO policy is a two hidden layer MLP of size [64, 64] with ReLU activations in between. These five conditions are summarised in table 1.

Table 1. Policy conditions evaluated in this study.

Condition	Input description	Dim	Notes
Raw observations	Flattened $7 \times 7 \times 3$ obs (integers)	147	Upper-bound baseline
OC-Agent	CLS-token VQ embedding only (e_{agent})	64	Ablation: no spatial or object context
OC-All	Concat $[e_{\text{agent}}, e_{\text{key}}, e_{\text{door}}]$	192	Three semantic tokens, no position
OC-All+Pos	Concat $[e_{\text{agent}}, e_{\text{key}}, e_{\text{door}}, \text{raw_xy}]$	194	Oracle diagnostic: float (x, y) appended
OC-All-Disc	Concat $[e_{\text{agent}}, e_{\text{key}}, e_{\text{door}}, e_{\text{pos}}]$	224	Proposed: fully discrete with pos embeddings

3.5 Evaluation

There are three evaluation metrics used in this study, each corresponding to three components of the experimental design: codebook quality, semantic alignment and policy learning.

The quality of the codebook is measured by using two criteria computed over the encoder’s training dataset. The active code count represents the number of codebook entries that are assigned to at least one observation in the dataset, entries with no assignments are considered collapsed and therefore inactive. Perplexity is calculated as $\exp(H)$ where H represents the entropy of the per-code usage distribution. Perplexity equal to K indicates that all K codes are used with an equal frequency, whereas a low perplexity represents that a small number of codes overwhelms the representation.

To measure the semantic alignment of the learned tokens, logistic regression probes are used [1]. For each of the VQ heads, the one-hot token index (K dimensional binary vector) is used as the feature for the logistic regression classifier

to predict each of the binary ground truth labels (`door_open`, `door_locked`, `carrying_key`). Probes are trained and fit on an 80/20 train-test split. Probe accuracy significantly above the majority baseline shows that the token assignments contain relevant information about their corresponding semantic concept. In addition to the per-head logistic probes there is a combined probe which is fit on the concatenation of the one-hot encodings for each of the three heads. The per-token purity is calculated as $P(\text{label} = \text{True} \mid \text{token} = k)$ for each code k , identifying individual codes that prove they act as hard semantic indicators.

The policy learning performance is measured by the mean episodic return, which is averaged across 10 deterministic test episodes run every 5,000 steps in the environment. Training continues for 200,000 total environment steps with the PPO policy [18]. Experiments are then repeated across 5 independent random seeds. Results are given as the mean $\pm$ standard deviation of the final evaluation run across the various seeds, measured alongside the mean peak return per seed and the bootstrap rate (the proportion of seeds that achieve a mean return above 0.5 at any point during training).

4 Results

4.1 Encoder Pre-Training

The object-centric encoder was trained for 80 epochs on the 59,697-observation dataset. Reconstruction loss converged across all heads, indicating that the Transformer backbone successfully learned to represent the observation content. Table 2 summarises the codebook utilisation for the final encoder, and Figure 1 shows the full training curves for losses and per-head perplexity.

Table 2. Codebook utilisation after 80 epochs of encoder pre-training.

VQ Head	K	Active codes	% active	Perplexity
Agent (CLS token)	64	40	62%	20.2
Key (cell readout)	16	16	100%	11.5
Door (cell readout)	8	8	100%	4.9

The key and door heads achieve full codebook usage, every code in those codebooks is used at least once and the perplexity of the key head (11.5/16) indicates quite well distributed usage of codes. The lower perplexity of the door head (4.9/8) reflects the binary nature of the door state (locked vs open), for which the 8 code vocabulary is able to represent with some dominant codes and low-frequency codes for more edge cases. The agent head is less utilised at 62% (40/64 active codes) which is expected as it likely reflects the non-uniform distribution of agent positions in the random policy dataset.

Figure 2 shows the token assignment distribution for each head, confirming full usage on the key and door heads and depicting the skewed but non-collapsed distribution of the agent head.

would a log-scale highlight some nuance here?

Why is there no more learning after 1-2 epochs?

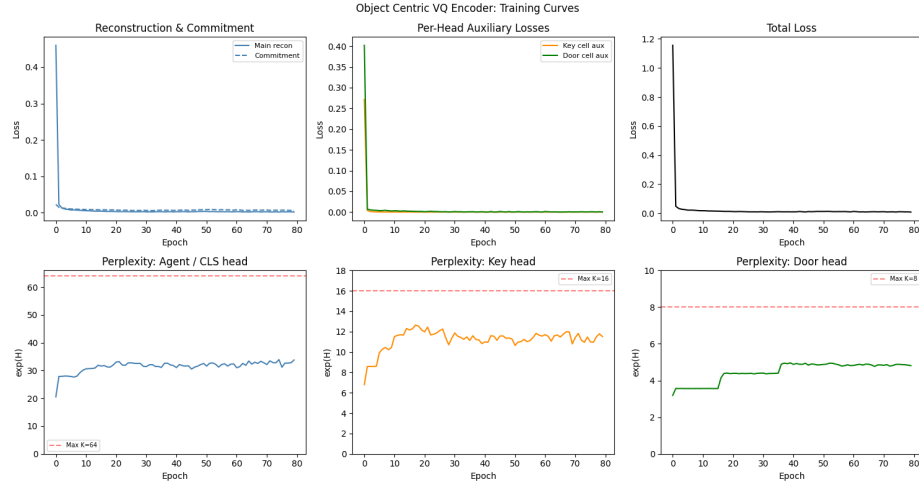


Fig. 1

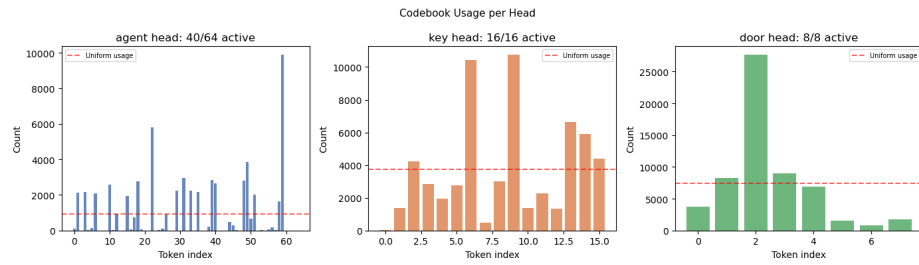


Fig. 2. Codebook usage per head. Bars depict the assignment count for one codebook vector over the full training dataset; the dashed red line marks uniform usage.

This result represents a significant improvement over the initial encoder prototype. The first design used a single CLS-token readout head with $K = 32$ and gradient-based codebook updates. After training, only 11 of 32 codes (34%) were active, with the remaining 21 codes never used. Replacing gradient-based updates with EMA codebook updates and adding dead-code restart raised active codes to 57/64. However, this approach still lead to issues during policy exploration as the spatial bottleneck was too significant as one CLS token could not represent the relevant agent history, positional information and semantic context. The subsequent redesign to an object-centric three-head architecture, with smaller per-object codebooks tuned to the scale of each concept ($K = 64$ for the agent, 16 for the key, 8 for the door), achieved the per-head codebook usage reported above.

4.2 Semantic Alignment

Table 3 outlines the logistic regression probe accuracy for each head/label combination, shown alongside the majority class baseline and the combined probe fit on the concatenated one-hot encodings of the three heads. Figure 3 visualises these results as grouped bar charts.

Table 3. Probe accuracy (logistic regression) for each VQ head and ground-truth label. Bold denotes the best per-head result for each label.

Label	Majority	Agent	Key	Door	Combined
door_open	0.944	0.970	0.949	0.972	0.985
door_locked	0.925	0.967	0.933	0.964	0.978
carrying_key	0.665	0.764	0.997	0.675	0.9997

The most significant result is the `carrying_key` label. The key head reaches a probe accuracy of 0.997, compared to the majority baseline represented in the dataset of 0.665, a gain of 33.2%. This near-perfect accuracy indicates that the key head’s token almost entirely determine the `carrying_key` state. Upon inspection of the per-token conditional probabilities: tokens 4,7,9 and 13 of the key head each have $P(\text{carrying_key} = \text{True}) = 1.000$. They appear solely when the agent is carrying the key. The other 12 key head tokens have $P(\text{carrying_key} = \text{True}) \approx 0.000$. This outlines a clear hard binary partition encoded into the codebooks inherent structure. The vocabulary has learned to spontaneously seperate into "key carried" and "key not carried" groups, with no semantically confusing ambiguous intermediate tokens.

The door head also achieves almost perfect semantic alignment for the door state labels. Token 5 of the door head is shown to have $P(\text{door_open} = \text{True}) = 1.000$ while all other tokens have $P(\text{door_open} = \text{True}) \approx 0.000$. The door head therefore has learned a single dedicated vocabulary for the open state, while the other remaining seven codes represent the locked state and it’s various sub-variants. The linear probe results confirm this finding has the accuracies

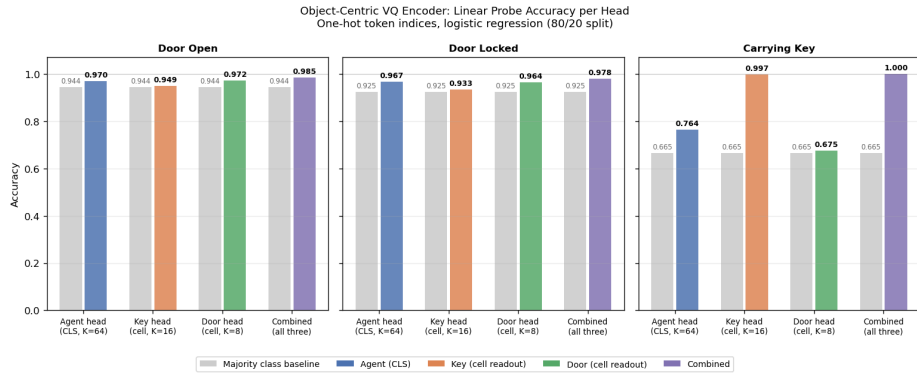


Fig. 3. Linear probe accuracy per VQ head for each ground-truth label. Grey bars represent the majority-class baseline. The key head (orange) achieves near-perfect accuracy for **carrying_key** (0.997), far exceeding the baseline of 0.665. The combined probe (purple) achieves the highest accuracy across all three labels.

are 0.972 for **door_open** and 0.964 for **door_locked**, meaningfully above the majority baselines of 0.944 and 0.925.

Figure 4 shows the per-token semantic purity plots for the key and door heads, making the hard binary partition directly visible. Outlining which tokens correspond to the various concepts.

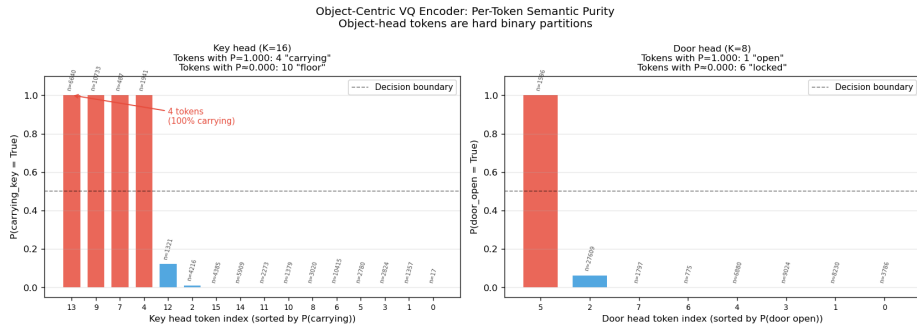


Fig. 4. Per-token semantic purity for the key and door head. Each bar shows $P(\text{label} = \text{True} \mid \text{token} = k)$ for tokens sorted by this probability. Red bars indicate tokens with $P = 1.000$, blue bars indicate tokens with $P \approx 0$.

The agent's head accuracy on **carrying_key** (0.764) is significantly lower than the key head's (0.997), despite the agent head having access to the same underlying representations and observation. This difference is explained by the architecture. The agent head takes the CLS token as input, which aggregates

information that is gathered from all 49 spatial positions through the Transformer’s attention mechanism. The key-carrying state, however, is a local event that occurs just at the key cell. This signal is diluted when it is pooled with the representations of all the other cells. The key head contrastingly directly reads the output of the Transformer at the key’s own spatial position, or activates the absent embedding when the key is not visible, which itself is its own reliable carrying signal. This comparison **outlines the value of the object-centric design, where global pooling loses the local semantic signal that direct object-cell readout maintains.**

in general: no need to repeat figures that are already in the table

The combined linear probe achieves 0.985 for `door_open`, 0.978 for `door_locked`, and 0.9997 for `carrying_key`, confirming that the three heads capture complementary and largely non-redundant information about the environment state.

4.3 Policy Learning Efficiency

Table 4 outlines the policy learning results for all five conditions, measured at **200,000** environment steps across five random seeds. Figure 5 shows the full learning curves.

Table 4. PPO policy learning results at 200k environment steps (5 seeds).

Condition	Dim	Mean return $\pm$ std	Bootstrap	Peak/seed
Raw observations	147	0.965 ± 0.001	5/5 ($\sim 30k$)	0.967
OC-Agent	64	0.000 ± 0.000	0/5	0.000
OC-All	192	0.116 ± 0.188	2/5	0.174
OC-All+Pos	194	0.307 ± 0.350	4/5	0.521
OC-All-Disc	224	0.768 ± 0.158	5/5 (45–85k)	0.945

I would suggest a step-down ordering in discussion for this table `OC-all-disc < OC-All-Pos < OC-All < OC-Agent < raw observations`

The ablation process reveals a clear outline about the information requirements for this task. Figure 5 outlines the policy learning across steps for each of the tested models.

OC-Agent (0.000 mean return, 0/5 seeds) fails completely: a single CLS-token semantic embedding without spatial information is unable to provide any useful signal for navigation. This is despite the fact that it is able to learn a useful embedding of the semantic concepts in the environment. OC-All (0.116 ± 0.188 , 2/5 seeds) improves slightly, however, still primarily fails at policy learning due to this same spatial bottleneck. OC-All (orange) is shown up to 500k steps to illustrate its slow, partial convergence and the spatial bottleneck that has been reached.

these results should be linked to the statement about information requirements above more explicitly

The issue here is that the **while both models are able to effectively create a discrete semantic embedding of the current state, these tokens can appear at different grid positions across episodes.** This creates contradictory and noisy policy gradients which leads to unstable learning.

this point needs to be developed more, I think you

OC-All+Pos (0.307 ± 0.350 , 4/5 seeds) adds the current agent position concatenated as two raw float values (x, y coordinates). This instantly creates a $2.6 \times$

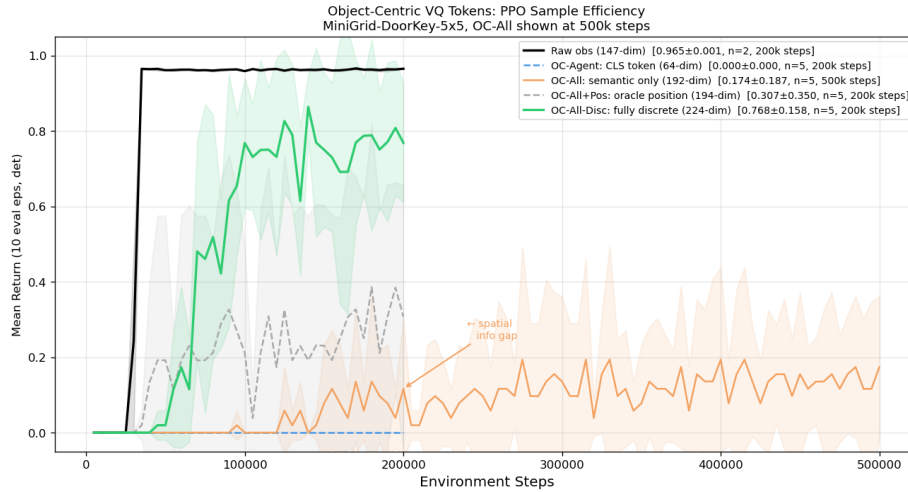


Fig. 5. PPO learning curves for all five conditions on MiniGrid-DoorKey-5x5. Mean return per episode plotted against environment steps, averaged over 5 seeds with shaded ± 1 standard deviation regions

improvement over OC-All which confirms the spatial bottleneck as the limiting factor to policy performance.

OC-All-Disc (0.768 ± 0.158 , 5/5 seeds, mean peak 0.945) is the proposed method and the best-performing non-raw condition. By replacing the two float coordinates with a 32-dimensional pre-trained positional embedding it yields a further $2.5\times$ improvement over the float condition with purely discrete inputs. Individual seeds reach peak returns of up to 0.970, approximately matching the raw-observation ceiling of 0.967.

The ablation process reveals a clear outline about the information requirements for this task. Figure 5 outlines the policy learning across steps for each of the tested models.

5 Discussion

add a brief intro as to what is in this chapter

5.1 Codebook Quality and the EMA Fix

The codebook collapse present in the initial gradient-based model, is a well-documented limitation of VQ training and clearly shows the tension between the expressiveness of the codebook and the stability of the optimisation process. In this setting, codebook values were updated as parameters via backpropagation. As the straight through estimator passes gradients through the **quantisation** step, the gradient signal received by a codebook entry is proportional to how often an entry is chosen. This positive feedback loop creates a situation where entries that are initialised to be close to encoder outputs are chosen often, influenced more

by gradient updates and move closer toward the mean of their cluster, making them more likely to be chosen by future iterations.

The EMA update rule [20] breaks this feedback loop which can create this instability by separating codebook updates from the gradient optimiser. Every entry to the codebook is replaced by a continual average of the encoders outputs that it is assigned, scaled by a decay hyperparameter ($\text{EMA_decay} = 0.99$ for this model). The dead-code restart mechanism also addresses this case where an entry is unused. It reinitialises these codes by copying a randomly modified version of an active entry’s current value, it then adds it back into the region of the codebook space that is occupied by the encoder. The use of these two mechanisms raised the active codes from 11/32 (34%) to 57/64 (89%). When re-designed to use object centric heads with individually sized codebooks maximum codebook utilisation was achieved by the key and door heads.

I missed the point of this section in relation to the RQ

5.2 Semantic Alignment

The semantic alignment results show strong evidence that a VQ encoder can learn semantically grounded representations. The first demonstration of these results was with the basic EMA-improved single CLS encoder which was able to encode `carrying_key` at a rate of 0.85 without explicit label supervision. These results were built upon by the introduction of the object centric model. The most compelling finding is the hard binary partition represented in the key head. Four codes specifically appear exclusively when the agent is carrying the key, and the remaining twelve appear only when the key is on the floor. This is a structural characteristic of the codebook, where two disjoint subsets of the vocabulary have been dedicated, through reconstruction-driven training alone, to the two values of a binary environment concept.

This hard binary partition is a direct consequence of the object-centric design decision, rather than global CLS pooling which, while able to encode `carrying_key` to approximately 0.85, produced softer statistical associations rather than the clean structural separation seen here. The key head reads the output of the transformer encoder at the key’s own spatial cell. Whenever the key is present on the floor, the cell carries a distinct representation, when the key is absent the head activates the learned absent embedding. This connects to the cognitive science motivation [2], that observations by human agents can efficiently encode complex continuous environments using compact, reusable representations. This is reflected here in the encoders emergence of a small, semantically clean vocabulary for each of the objects states.

The probe accuracy for the door-related labels (0.970–0.985) have less of a striking improvement over the baseline, largely due to the fact that the baseline values are very high (0.925–0.944). Therefore, reflecting the infrequent nature of the door-open state in the random-policy dataset. Nonetheless, the information gain above the baseline is meaningful as the combined probe achieves 0.985 for `door_open` compared to a 0.944 baseline, indicating that the tokens carry real additional information about door state.

5.3 Policy Learning

The results of the policy learning process are best interpreted through the ablation process, which outlines the spatial information gap as the primary limitation to generalizable policy learning, and clearly outlines the positional embedding as the way of closing it. OC-Agent’s failure (0.000 mean return) shows that semantic token quality alone is not enough for reliable navigation. OC-All’s partial success (0.116, 2/5 seeds) demonstrates that three factored semantic tokens help with the perception of the environment, but are still unable to largely overcome the challenges of the spatial bottleneck. The concatenated coordinates approach (OC-All+Pos, 0.307) helps to isolate the effect of positional information by providing the ground truth (x, y) coordinates.

The fact that OC-All-Disc (0.768) outperforms OC-All+Pos (0.307) is noteworthy. Two raw float values provide less useful signal to an MLP than a 32-dimensional pre-trained distributed embedding for several reasons: the embedding has higher feature dimensionality, encoding more gradient-accessible structure; the pre-training on reconstruction data ensures that nearby positions receive nearby embeddings, encoding a smooth spatial prior; and the scale of the embedding dimensions is compatible with the VQ embeddings it is concatenated with. This result suggests that the quality of spatial encoding matters, not merely its presence.

Another relevant learning is that OC-All-Disc (0.768) significantly outperforms OC-All+Pos (0.307). This makes it clear that two raw float continuous values provide much less useful signal than a 32 dimensional pre-trained distributed embedding to an MLP. This is likely due to several reasons. The embedding has a higher feature dimensionality, which can encode more gradient-accessible structure, the pre-training process on reconstruction data helps to ensure that nearby positions receive nearby embeddings, encoding a smooth spatial prior. Lastly, the scale of the embedding dimensions is compatible with the VQ embeddings it is concatenated with. This result indicates that the quality of the spatial encoding is relevant over purely it’s presence in the architecture.

The instability observed during training for all models, particularly OC-All-Disc (seeds that bootstrap and reach near optimal performance before oscillating downward), is consistent with the sensitivity of PPO policies to it’s learning rate in sparse-reward environments. The implementation of a cosine or step-learning rate schedule may likely improve convergence.

5.4 Architecture Progression

The final encoder architecture resulted from a sequence of principled design decisions, each motivated by a specific empirical observation. The first encoder prototype used a single CLS-token VQ readout ($K = 32$) with gradient-based codebook updates. The problem that emerged was codebook collapse (11/32 active codes). The object-centric redesign was motivated by the probe results on the CLS-token encoder: the agent head’s `carrying_key` probe accuracy was substantially below what seemed achievable given the binary nature of the label.

The hypothesis was that direct object-cell readout would recover the local signal lost by CLS pooling. This hypothesis was confirmed by the key head’s 0.997 probe accuracy.

The final encoder architecture is a result of a series of principled individual design decisions, which are each motivated by a specific empirical observation. The initial encoder prototype used a single CLS-token VQ model ($K = 32$) with gradient-based codebook updates. This created codebook collapse. The EMA-fixed single-head encoder improved `carrying_key` probe accuracy from 0.667 to approximately 0.85, confirming that codebook health was a prerequisite for semantic alignment but that global CLS pooling imposed a ceiling and a challenge for policy learning, resulting in the object-centric redesign. The hypothesis was that localized, direct object-cell readout would help to recover the local signal lost by CLS pooling. This hypothesis was confirmed by the key head’s 0.997 probe accuracy.

The positional embedding design was motivated by the policy learning study. The collapse of the OC-Agent demonstrated the need for spatial information while the spatial ablation of OC-All+Pos confirmed that float coordinates were insufficient for strong performance, and lost the discrete nature of the study. The `PositionEmbeddingEncoder` was introduced as a pre-trainable, discrete alternative.

5.5 Limitations

There are several limitations that constrain the conclusion that can be drawn from this body of work. Firstly, experiments were limited to a single small environment (MiniGrid-DoorKey-5x5). The question still remains to be answered whether or not similar semantic alignment properties observed here would persist in larger most complex observation spaces with different types of objects. Secondly, the policy learning comparison does **still** show that general raw pixel observations does **still** perform better in terms of sample efficiency and overall performance to this discrete approach. Third, policy learning under OC-All-Disc demonstrates significant instability, possibly attributable to the absence of a learning rate schedule.

5.6 Future Work

These results suggest several productive directions for future study. The first immediate extension would be to extend these frameworks for semantic alignment to larger and more complex environments, where the raw observation is less compact and the type of different semantic states is much larger.

A second direction is to research whether planning over the discrete token vocabulary provides a meaningful advantage over these policy gradient methods. For the purpose of this study a PPO policy using an MLP was chosen to provide a meaningful baseline, however, there is no inherent property of PPO that is able to take advantage of these compressed representations. The IRIS architecture [13] provides a natural framework for this extension, with the object-centric encoder

replacing the VQ-VAE tokeniser. Where world models or some other type of learning algorithm can operate over this compressed token space.

Thirdly, neurosymbolic approaches may be particularly powerful for this application. Given that codebook tokens correspond to meaningful, interpretable state concepts they could serve as primitives for downstream symbolic reasoning. Integration with a logic program or planning algorithm could operate over these sequences of tokens rather than raw observations.

develop more

Fourth, end-to-end training of the entire pipeline (encoder, positional embedding and policy) could jointly be explored with improved regularisation to maintain semantic alignment. Finally, multi-task generalisation experiments, training a single encoder on one MiniGrid task and measuring the learned vocabulary’s transferability to a related task could test whether these semantically grounded discrete tokens support cross-task generalisation.

6 Conclusion

This thesis covered the question to which degree discrete, Transformer-based state encoders can learn semantically meaningful representations in continuous reinforcement learning environments.

Regarding codebook health, the object-centric VQ encoder with EMA codebook updates and dead-code restart achieved full utilisation on the key and door heads (100% active codes) and 62% utilisation on the agent head, compared to 34% under the gradient-based baseline. The EMA mechanism is effective at preventing codebook collapse in RL observation datasets.

In reference to codebook health, the object VQ encoder with EMA codebook updates and dead-code restarts achieved maximum utilisation on the key and door heads (100% codes) and 62% utilisation on the agent’s head, compared to 34% under the gradient-based approach baseline.

Regarding semantic alignment, the key head achieved an accuracy of 0.997 using a linear probe for the carrying_key label, with four distinct tokens demonstrating clear semantic boundaries (P (

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Alain, G., Bengio, Y.: Understanding intermediate layers using linear classifier probes. ICLR Workshop (2017)
2. Fang, Z., Sims, C.R.: Humans learn generalizable representations through efficient coding. Nature Communications **16**(1), 3989 (2025). <https://doi.org/10.1038/s41467-025-58848-6>
3. Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., Davidson, J.: Learning latent dynamics for planning from pixels. ICML (2019)
4. Hafner, D., Lillicrap, T., Norouzi, M., Ba, J.: Dream to control: Learning behaviors by latent imagination. ICLR (2020)

5. Hafner, D., Pasukonis, J., Ba, J., Lillicrap, T.: Mastering diverse domains through world models. [arXiv:2301.04104](#) (2023)
6. Hansen, N., Su, H., Wang, X.: TD-MPC2: Scalable, robust world models for continuous control. ICLR (2024)
7. Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S., Lerchner, A.: beta-VAE: Learning basic visual concepts with a constrained variational framework. ICLR (2017)
8. Janner, M., Li, Q., Levine, S.: Offline reinforcement learning as one big sequence modeling problem. NeurIPS (2021)
9. Kaiser, L., et al.: Model-based reinforcement learning for Atari. ICLR (2019)
10. Koh, P.W., et al.: Concept bottleneck models. ICML (2020)
11. Levine, S., Finn, C., Darrell, T., Abbeel, P.: End-to-end training of deep visuomotor policies. JMLR (2016)
12. Mentzer, F., Foster, D., Minnen, D., Agustsson, E., Toderici, G.: Finite scalar quantization: VQ-VAE made simple. [arXiv:2309.15505](#) (2023)
13. Micheli, V., Alonso, E., Fleuret, F.: Transformers are sample-efficient world models. ICLR (2023)
14. Chevalier-Boisvert, M., et al.: Minigrid & Miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. NeurIPS Datasets and Benchmarks (2023)
15. Mnih, V., et al.: Human-level control through deep reinforcement learning. *Nature* **518**, 529–533 (2015)
16. Razavi, A., van den Oord, A., Vinyals, O.: Generating diverse high-fidelity images with VQ-VAE-2. NeurIPS (2019)
17. Reed, S., et al.: A generalist agent. TMLR (2022)
18. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. [arXiv:1707.06347](#) (2017)
19. Doshi-Velez, F., Kim, B.: Towards a rigorous science of interpretable machine learning. [arXiv:1702.08608](#) (2017)
20. van den Oord, A., Vinyals, O., Kavukcuoglu, K.: Neural discrete representation learning. NeurIPS (2017)

A Hyperparameter Tables

Table 5 lists the hyperparameters used for encoder pre-training. Table 6 lists the PPO hyperparameters used for policy learning across all conditions.

Table 5. Encoder pre-training hyperparameters.

Hyperparameter	Value
Batch size	128
Epochs	80
Learning rate (Adam)	3×10^{-3}
Commitment loss coefficient (β)	0.10
EMA decay	0.99
Dead-code threshold	1.0
Codebook size K (agent head)	64
Codebook size K (key head)	16
Codebook size K (door head)	8
Embedding dimension D_{model}	64
Transformer heads	4
Transformer layers	2
Position embedding dimension	32

Table 6. PPO policy learning hyperparameters (all conditions).

Hyperparameter	Value
n_steps	512
Batch size	64
n_epochs (per rollout)	4
Discount factor γ	0.99
GAE lambda	0.95
Clip epsilon	0.2
Entropy coefficient	0.05
Value function coefficient	0.5
Max gradient norm	0.5
Learning rate	3×10^{-3}
Total environment steps	200,000
Evaluation frequency (steps)	5,000
Evaluation episodes per checkpoint	10
Number of seeds	5
Policy net architecture	[64, 64] MLP, ReLU